



Batch: C3 Roll No.: 16010123325

Experiment / Assignment / Tutorial No. 4

Title: Implementation of PCA for Dimensionality Reduction in Correlated Datasets.

Aim: To understand & implement PCA for dimensionality reduction using python.

Expected Outcome of Experiment:

CO3: Employ unsupervised learning methods to explore, categorize data and differentiate patterns within datasets.

Books/ Journals/ Websites referred:

1. "Machine Learning", Tom M. Mitchell, McGraw Hill, 2017.
2. <https://builtin.com/data-science/step-step-explanation-principal-component-analysis>
3. <https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/>
4. Python Data Science Essentials - Third Edition, 2018 Packt Publishing

Theory:

Dimensionality Reduction

Dimensionality reduction is the process of reducing the number of input variables (features) in a dataset while preserving the essential structure and information.

Reasons for Dimensionality Reduction

- High-dimensional datasets suffer from **data sparsity**, reducing prediction reliability.
- Increased computational cost and memory usage.
- Redundant and correlated features do not add new information.
- Visualization becomes difficult when the number of features increases.

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is an **unsupervised statistical technique** used for dimensionality reduction. PCA transforms the original correlated features into a

new set of **orthogonal (uncorrelated) variables** called **principal components**, arranged in decreasing order of variance.

Steps to implement Principal Component Analysis (PCA)

Step 1: Standardization and Centering Data

Centering data is to standardize the range of the continuous initial variables so as to ensure equal contribution by each variable to the PCA analysis to avoid biased results. This can be done by subtracting the mean and dividing by the standard deviation for each value of each variable.

$$z = \frac{\text{value} - \text{mean}}{\text{standard deviation}}$$

Step 2: Computation of Covariance Matrix

In the **covariance matrix between vector with itself**, the matrix of covariances between the scalar components of the vector **X**

$$\text{var}(\mathbf{X}) = \text{Cov}(\mathbf{X}, \mathbf{X}) = E[(\mathbf{X} - E[\mathbf{X}])(\mathbf{X} - E[\mathbf{X}])^T]$$

The cross-covariance between two vectors is

$$\text{Cov}(\mathbf{X}, \mathbf{Y}) = \mathbf{K}_{\mathbf{XY}} = E[(\mathbf{X} - E[\mathbf{X}])(\mathbf{Y} - E[\mathbf{Y}])^T]$$

The covariance matrix is a $N \times N$ symmetric matrix (where N is the number of dimensions (variables))

For example, for a 3-dimensional data set with 3 variables x , y , and z , the covariance matrix is a 3×3 data matrix of this form:

$$\begin{bmatrix} \text{Cov}(x, x) & \text{Cov}(x, y) & \text{Cov}(x, z) \\ \text{Cov}(y, x) & \text{Cov}(y, y) & \text{Cov}(y, z) \\ \text{Cov}(z, x) & \text{Cov}(z, y) & \text{Cov}(z, z) \end{bmatrix}$$

The covariance is commutative ($\text{Cov}(a, b) = \text{Cov}(b, a)$). Hence, the entries of the covariance matrix are symmetric with respect to the main diagonal, which means that the upper and the lower triangular portions are equal.

- If covariance value between two variables is positive then the two variables increase or decrease together (they are correlated)
- If covariance value between two variables is negative then as one variable increases, the other decreases (they are inversely correlated)

Step 3: Eigen Decomposition and Identifying Principal Components

Eigenvectors and eigenvalues are to be computed from the covariance matrix in order to determine the **principal components** of the data.

The eigenvectors of the Covariance matrix are the directions of the axes where there is the most variance (most information) and that are called as Principal Components. And eigenvalues are simply the coefficients of the eigenvectors, which give the amount of variance carried in each Principal Component. By ranking your eigenvectors in order of their eigenvalues, highest to lowest, you get the principal components in order of significance.

Principal Component Analysis Example:

Let's suppose that our data set is 2-dimensional with 2 variables **x and y** and that the eigenvectors and eigenvalues of the covariance matrix are as below:

$$v1 = \begin{bmatrix} 0.6778736 \\ 0.7351785 \end{bmatrix} \quad \lambda_1 = 1.284028$$

$$v2 = \begin{bmatrix} -0.7351785 \\ 0.6778736 \end{bmatrix} \quad \lambda_2 = 0.04908323$$

If we rank the eigenvalues in descending order, we get $\lambda_1 > \lambda_2$, which means that the eigenvector that corresponds to the first principal component (PC1) is $v1$ and the one that corresponds to the second principal component (PC2) is $v2$.

The percentage of variance of the principal components is computed (information) for each component, by dividing the eigenvalue of each component by the sum of eigenvalues. If we apply this on the example above, we find that PC1 and PC2 carry respectively 96 % and 4 % of the variance of the data.

Step 4: Feature Selection and Creating a Feature Vector

Computing the eigenvectors and ordering them by their eigenvalues in descending order is needed to find the principal components in order of significance.

The feature vector is simply a matrix that has as columns the eigenvectors of the components that we decide to keep. This makes it the first step towards dimensionality reduction, because if we choose to keep only p eigenvectors (components) out of n , the final data set will have only p dimensions.

Principal Component Analysis Example:

Continuation of the example from the previous step, we can either form a feature vector with both of the eigenvectors $v1$ and $v2$:

$$\begin{bmatrix} 0.6778736 & -0.7351785 \\ 0.7351785 & 0.6778736 \end{bmatrix}$$

Or discard the eigenvector $v2$, which is the one of lesser significance, and form a feature vector with $v1$ only:

$$\begin{bmatrix} 0.6778736 \\ 0.7351785 \end{bmatrix}$$

Discarding the eigenvector $v2$ will reduce dimensionality by 1, and will consequently cause a loss of information in the final data set. But as $v2$ is carrying only 4 percent of the information, the loss will be less important and we will still have 96 percent of the information that is carried by $v1$.

It is up to you to decide whether to keep all the components or discard the ones of lesser significance, depending on the application and required outcome.

Step 5: Data Projection

In this step, the aim is to use the feature vector formed using the eigenvectors of the covariance matrix, to reorient the data from the original axes to the ones represented by the principal components (hence the name Principal Components Analysis). This can be done by multiplying the transpose of the original data set by the transpose of the feature vector.

$$FinalDataSet = FeatureVector^T * StandardizedOriginalDataSet^T$$

Principal Component Analysis Variants

- 1. Kernel PCA (KPCA)** - Captures non-linear relationships between features.
- 2. Sparse PCA (SPCA)** - Creates interpretable components and mitigates overfitting in high-dimensional data.
- 3. Robust PCA (RPCA)** - Handles outliers and noise by separating them from the true low-dimensional structure.



Department of Computer Engineering

4. Incremental PCA (IPCA) - Scalable to large data sets and suitable for real-time data tasks.

5. Probabilistic PCA (PPCA) - Provides a probabilistic interpretation of data and supports inference under uncertainty.

Problem Definition:

Apply **Principal Component Analysis (PCA)** on a real-world dataset (Pizza dataset / Diabetes dataset / any correlated numerical dataset) to reduce dimensionality and analyze the variance preserved by the selected principal components.

Reference Link for dataset

<https://drive.google.com/drive/folders/1GxcvBAWrdwemDrxWBhjWEayCrcXZGaft?usp=sharing>

Details of data set used:

<https://drive.google.com/drive/folders/1GxcvBAWrdwemDrxWBhjWEayCrcXZGaft?usp=sharing>

API/Function used:

- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Functions:

- `pandas.read_csv()` - Load dataset
- `StandardScaler()` - Standardize data
- `PCA()` - Apply PCA
- `fit_transform()` - Transform dataset
- `explained_variance_ratio_` - Check variance retained
- `matplotlib.pyplot` - Visualization

Implementation:

Pizza Dataset

```
import pandas as pd
import numpy as np
import time
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

data = pd.read_csv("pizza.csv")

# Target variable
y = data["Brand"]

# Features
X = data.drop("Brand", axis=1)

# 2. Feature Scaling
# (Important for PCA)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

# 4. Normal Classification
start_normal = time.time()

model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy_normal = accuracy_score(y_test, y_pred)

end_normal = time.time()

time_normal = end_normal - start_normal

# 5. Apply PCA
```

```
pca = PCA()

X_pca = pca.fit_transform(X_scaled)

print("\nPrincipal Components:\n")
print(pca.components_)

print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)

# 6. Reduce Dimensions
pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

# 7. Train-Test Split After PCA
X_train_pca, X_test_pca, y_train, y_test = train_test_split(
    X_pca, y, test_size=0.2, random_state=42
)

# 8. Classification After PCA
start_pca = time.time()

model = LogisticRegression(max_iter=200)
model.fit(X_train_pca, y_train)

y_pred_pca = model.predict(X_test_pca)

accuracy_pca = accuracy_score(y_test, y_pred_pca)

end_pca = time.time()

time_pca = end_pca - start_pca

# 9. Results Comparison
print("\n----- Results Comparison -----")

print("Accuracy without PCA:", accuracy_normal)
print("Time without PCA:", time_normal)

print("\nAccuracy with PCA:", accuracy_pca)
print("Time with PCA:", time_pca)
```

Diabetes Dataset

```
import pandas as pd
import numpy as np
import time

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

data = pd.read_csv("diabetes.csv")

# Target variable
y = data["Outcome"]

# Features
X = data.drop("Outcome", axis=1)

# 2. Feature Scaling
# (Important for PCA)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

# 4. Normal Classification
start_normal = time.time()

model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy_normal = accuracy_score(y_test, y_pred)

end_normal = time.time()

time_normal = end_normal - start_normal
```

```
# 5. Apply PCA
pca = PCA()

X_pca = pca.fit_transform(X_scaled)

print("\nPrincipal Components:\n")
print(pca.components_)

print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)

# 6. Reduce Dimensions
pca = PCA(n_components=0.5)

X_pca = pca.fit_transform(X_scaled)

# 7. Train-Test Split After PCA
X_train_pca, X_test_pca, y_train, y_test = train_test_split(
    X_pca, y, test_size=0.2, random_state=42
)

# 8. Classification After PCA
start_pca = time.time()

model = LogisticRegression(max_iter=200)
model.fit(X_train_pca, y_train)

y_pred_pca = model.predict(X_test_pca)

accuracy_pca = accuracy_score(y_test, y_pred_pca)

end_pca = time.time()

time_pca = end_pca - start_pca

# 9. Results Comparison
print("\n----- Results Comparison -----")

print("Accuracy without PCA:", accuracy_normal)
print("Time without PCA:", time_normal)
```



```
print("\nAccuracy with PCA:", accuracy_pca)
print("Time with PCA:", time_pca)
```

Results:

Pizza Dataset

```
----- Results Comparison -----
Accuracy without PCA: 0.8166666666666667
Time without PCA: 0.023710966110229492

Accuracy with PCA: 0.7166666666666667
Time with PCA: 0.00455021858215332
shreya@Shreyas-MacBook-Air-2 ML %
shreya@Shreyas-MacBook-Air-2 ML %
shreya@Shreyas-MacBook-Air-2 ML %
shreya@Shreyas-MacBook-Air-2 ML % python3 exp4.py

Principal Components:

[[-1.99403012e-02  6.44785509e-02  3.78951071e-01  4.46467403e-01
  4.71813825e-01  4.35550982e-01 -4.24755778e-01  2.44494235e-01]
 [ 5.81763147e-03  6.28375256e-01  2.69609639e-01 -2.34215299e-01
  1.11065055e-01 -2.01521828e-01 -3.20416546e-01 -5.67407501e-01]
 [ 9.93250052e-01  4.89635165e-02 -8.22473744e-02  4.52319845e-02
  3.45464203e-04  4.33743722e-02 -1.89923636e-02  2.04451116e-03]
 [ 1.11331687e-01 -4.17773426e-01  7.40512315e-01 -1.93036121e-01
  5.57179039e-02 -4.56740689e-01  4.90903283e-02  1.15424367e-01]
 [ 2.07984566e-02 -2.22559480e-01 -8.86180970e-03 -5.06869519e-01
  5.55431191e-01  4.41865186e-01  3.34420063e-01 -2.78305646e-01]
 [-1.39813075e-02 -4.55739129e-03 -3.89773631e-01  1.75580059e-01
  6.68467507e-01 -6.03270611e-01  6.06485193e-03  7.88092216e-02]
 [ 2.58398640e-04  4.46453904e-01 -6.96421592e-05 -5.25427705e-01
  5.88126045e-02  3.21259514e-03 -4.32498538e-04  7.21896902e-01]
 [-9.96706744e-05  4.18545592e-01  2.76733787e-01  3.77732492e-01
  5.60368303e-02 -5.55686523e-04  7.76061498e-01  1.20112159e-02]]

Explained Variance Ratio:
[5.21630440e-01 2.86312631e-01 1.25839429e-01 5.08893635e-02
 1.18484671e-02 3.43628701e-03 4.21930078e-05 1.18863298e-06]

----- Results Comparison -----
Accuracy without PCA: 0.8166666666666667
Time without PCA: 0.005738973617553711

Accuracy with PCA: 0.4333333333333333
Time with PCA: 0.0032660961151123047
shreya@Shreyas-MacBook-Air-2 ML %
```

The first few principal components capture most of the dataset variance ($\approx 80\%$ using 2 PCs, $\approx 93\%$ using 3 PCs). PCA reduces dimensionality while retaining significant information. Classification accuracy remains comparable, demonstrating that PCA effectively compresses the dataset with minimal loss in performance while slightly improving computational efficiency.

Diabetes Dataset

```
shreya@Shreyas-MacBook-Air-2 ML % python3 pca_diabetes.p
y
Principal Components:

[[ 0.1284321  0.39308257  0.36000261  0.43982428  0.43502617  0.45194134
   0.27061144  0.19802707]
 [ 0.59378583  0.17402908  0.18389207 -0.33196534 -0.25078106 -0.1009598
  -0.122069  0.62058853]
 [ 0.01308692 -0.46792282  0.53549442  0.2376738 -0.33670893  0.36186463
  -0.43318905 -0.07524755]
 [ 0.08069115 -0.40432871  0.05598649  0.03797608 -0.34994376  0.05364595
   0.8336801  0.0712006 ]
 [ 0.47560573 -0.46632804 -0.32795306  0.48786206  0.34693481 -0.25320376
  -0.11981049  0.10928996]
 [ 0.19359817  0.09416176 -0.6341159  0.00958944 -0.27065061  0.68537218
  -0.08578409 -0.03335717]
 [-0.58879003 -0.06015291 -0.19211793  0.28221253 -0.13200992 -0.03536644
  -0.08609107  0.71208542]
 [ 0.11784098  0.45035526 -0.01129554  0.5662838 -0.54862138 -0.34151764
  -0.00825873 -0.21166198]]

Explained Variance Ratio:
[0.26179749 0.21640127 0.12870373 0.10944113 0.09529305 0.08532855
 0.05247702 0.05055776]

----- Results Comparison -----
Accuracy without PCA: 0.7532467532467533
Time without PCA: 0.004023075103759766

Accuracy with PCA: 0.7272727272727273
Time with PCA: 0.00127410888671875
shreya@Shreyas-MacBook-Air-2 ML %
```

PCA was used to reduce the dimensionality of the diabetes dataset. The variance is distributed across multiple components, with the first few capturing a moderate amount of information. After applying PCA, accuracy slightly decreased from 75.32% to 72.73%, showing minor information loss. However, computation time was significantly reduced (about 3× faster), demonstrating improved efficiency. Thus, PCA offers a trade-off between accuracy and speed for high-dimensional data.

Conclusion:

PCA effectively reduced the dimensionality of the dataset while preserving most of the important information. It helped eliminate redundancy due to correlated features and improved data visualization. This demonstrates how unsupervised learning techniques can be used to explore and understand complex datasets.



Post Lab:

1. What is the role of eigenvalues and eigenvectors in PCA? How do they help in dimensionality reduction?

Ans- In PCA, eigenvalues and eigenvectors come from the covariance matrix, and they are the core of dimensionality reduction.

- Eigenvectors - represent the directions (principal components) in which the data varies the most
- Eigenvalues - represent the amount of variance along those directions

So basically:

- The eigenvector with the largest eigenvalue = most important direction (PC1)
- The next = PC2, and so on
- How this helps in dimensionality reduction:
- You keep only top k eigenvectors (with highest eigenvalues)
- These retain most of the data's variance
- You discard the rest, reducing dimensions with minimal information loss

2. Given the dataset:

$$X = \{(2,1), (3,5), (4,3), (5,6), (6,7), (7,8)\}$$

- Compute the **covariance matrix** of the data.
- Find the **eigenvalues** and **eigenvectors** of this covariance matrix.

ML Postlab.

Given: $X = \{(2,1), (3,5), (4,3), (5,6), (6,7), (7,8)\}$.

Mean:

$$\bar{x} = \frac{2+3+4+5+6+7}{6} = 4.5$$

$$\bar{y} = \frac{1+5+3+6+7+8}{6} = 5$$

Center data:

x	y	$x-4.5$	$y-5$
2	1	-2.5	-4
3	5	-1.5	0
4	3	-0.5	-2
5	6	0.5	1
6	7	1.5	2
7	8	2.5	3

$$\text{Cov}(X,Y) = \frac{1}{n-1} \sum (x_i - \bar{x})(y_i - \bar{y})$$

$\Rightarrow \text{Cov}(X,X) = 3.5$
 $\Rightarrow \text{Cov}(Y,Y) = 6.8$
 $\Rightarrow \text{Cov}(X,Y) = 4.6$

$\therefore \text{Cov. Mat.} = \begin{bmatrix} 3.5 & 4.6 \\ 4.6 & 6.8 \end{bmatrix}$

$|A - \lambda I| = 0$

$\Rightarrow \lambda_1 \approx 10.04$ & $\lambda_2 \approx 0.26$

$$\begin{aligned}\Rightarrow \text{Cov}(X, X) &= 3.5 \\ \Rightarrow \text{Cov}(Y, Y) &= 6.8 \\ \Rightarrow \text{Cov}(X, Y) &= 4.6 \\ \therefore \text{Cov. Mat.} &= \begin{bmatrix} 3.5 & 4.6 \\ 4.6 & 6.8 \end{bmatrix} \\ |A - \lambda I| &= 0 \\ \Rightarrow \lambda_1 &\approx 10.04 \text{ \& } \lambda_2 \approx 0.26 \\ \text{For } \lambda_1 &= 10.04: \\ \text{Eig. Vector} &= \begin{bmatrix} 0.56 \\ 0.83 \end{bmatrix} \\ \text{For } \lambda_2 &= 0.26: \\ \text{Eig. Vector} &= \begin{bmatrix} -0.83 \\ 0.56 \end{bmatrix} \\ PC1 &\approx 10.04 \Rightarrow \text{Captures } \sim 97\% \text{ variance.} \\ \text{Second contributor} &\text{ very little } \Rightarrow \sim 3\%. \\ \therefore \text{We can reduce 2D} &\rightarrow \text{1D by keeping only} \\ &\text{PC1 with minimal info. loss.}\end{aligned}$$